```
one:
  title: MyString
  description: MyText
  image_url: MyString
  price: 9.99

two:
  title: MyString
  description: MyText
  image_url: MyString
  price: 9.99
```

The fixture file contains an entry for each row that we want to insert into the database. Each row is given a name. In the case of the Rails-generated fixture, the rows are named `one` and `two`. This name has no significance as far as the database is concerned—it isn't inserted into the row data. Instead, as you'll see shortly, the name gives us a convenient way to reference test data inside our test code. They also are the names used in the generated integration tests, so for now, we'll leave them alone.



David says:

Picking Good Fixture Names

As with the names of variables in general, you want to keep the names of fixtures as self-explanatory as possible. This increases the readability of the tests when you're asserting that `product(:valid_order_for_fred)` is indeed Fred's valid order. It also makes it a lot easier to remember which fixture you're supposed to test against, without having to look up `p1` or `order4`. The more fixtures you get, the more important it is to pick good fixture names. So starting early keeps you happy later.

But what do we do with fixtures that can't easily get a self-explanatory name like `valid_order_for_fred`? Pick natural names that you have an easier time associating to a role. For example, instead of using `order1`, use `christmas_order`. Instead of `customer1`, use `fred`. Once you get into the habit of natural names, you'll soon be weaving a nice little story about how `fred` is paying for his `christmas_order` with his `invalid_credit_card` first, then paying with his `valid_credit_card`, and finally choosing to ship it all off to `aunt_mary`.

Association-based stories are key to remembering large worlds of fixtures with ease.

Inside each entry you can see an indented list of name-value pairs. As in your